08 / ARCHITECTURE REVIEW

A strong compiler foundation. *Uneven guarantees around it.*

The main opportunity is to make correctness, identity, state and evaluation consistent across the service. The project does not need a broad microservice expansion to address its most consequential weaknesses.

Snapshot `9f71aa300421775e4b87ff1ef3b6e202c983bc87`
Review date: 09 November 2026 · Conclusions refer to this checkout.

DESIGN SUMMARY

The architecture is sensible in outline.

The frontend delegates over HTTP to a Python service. The service prefers deterministic translation, can retrieve saved examples, and can ask Gemini for fallback or repair. Local validators, live execution and optional result comparison aim to increase trust. Feedback can feed the reference corpus, cached context and model training.

The compiler extraction into typed IR, data-driven resolution and separate emitters is the clearest architectural strength. The weakest areas are the contracts between those layers: a review flag can be lost, missing validation can become success, and tenant-specific results can enter shared caches. The learning architecture is less complete than the synchronous translator.

Skip to content

Preserve these strengths

- Deterministic compiler and typed query representation.
- Mapping data separated from language logic.
- HTTP separation between Streamlit and backend.
- Dependency injection and typed translation context as test seams.
- Authentication middleware, admin checks, encrypted credential support, query guards and TLS verification.
- Infrastructure intent: workload identities, Secret Manager, private VMs and explicit network controls.
- A substantial local suite, including a passing compiler-specific suite.

Improve these design properties

- Truthful validation states and preserved semantic uncertainty.
- Tenant-scoped execution evidence and versioned caches.
- One request/result contract across all API variants.
- Durable ownership of feedback, jobs and knowledge revisions.
- Independent evaluation with enforceable dataset isolation.
- Executable deployment and training contracts.
- Smaller orchestration responsibilities and observable failures.

ARCHITECTURAL TRADEOFFS

Keep the core cohesive; separate work by lifecycle.

DECISION	BENEFIT	COST / BOUNDARY TO RESPECT
Modular translation backend	Low-latency in-process compiler, resolver and validation calls; straightforward local tests.	Modules need typed contracts and explicit state ownership to avoid hidden coupling.
Deterministic first, AI when needed	Inspectable behavior for known constructs; flexibility for long-tail queries.	AI output must preserve constraints and receive the same evidence requirements.

Skip to content

DECISION	BENEFIT	COST / BOUNDARY TO RESPECT
Saved gold examples	Fast, repeatable answers for known queries.	Literal-preserving keys, provenance and independent held-out evaluation are essential.
Asynchronous learning workers	Training and corpus maintenance can run independently of request latency.	Requires durable artifacts, idempotent jobs and explicit promotion authority.
Live verification	Checks a real target environment.	Tenant identity, cost, time window, query limits and absent evidence must be explicit.

Overall assessment

Useful and substantial engineering, especially in the compiler. The current checkout should not be treated as an end-to-end verified, automatically self-improving translation platform. The highest-value improvements strengthen the guarantees already advertised by the design.

EVIDENCE-BACKED FINDINGS

Where the design currently breaks its promises.

Severity reflects review judgment about impact, not an official vulnerability score. “Reproduced” means exercised locally with synthetic inputs or test doubles; it does not mean a production incident was observed.

Skip to content

HIGH REPRODUCED F01

A live check can report success without running

With no Cortex credentials, _check_cortex_execution returns an empty error list. The orchestrator interprets that as live_validation_passed=true. Exceptions also become empty error lists. The HTTP translation routes explicitly disable parity even when requested.

WHY IT MATTERS

A user can receive syntactically valid output carrying stronger verification signals than the evidence supports. Local parsing, live execution and source/target equivalence are different claims.

IMPROVEMENT PRINCIPLE

Make validation evidence explicit: not requested, unavailable, failed, or passed, with the environment and query revision attached. Preserve compiler uncertainty through every adapter.

▶ **Source evidence**

Skip to content

HIGH REPRODUCED F02

Shared caches cross tenant and execution-policy boundaries

The full-result cache key excludes tenant credentials, disable_llm, shadow mode and model/corpus revisions. A second request with different tenant credentials returned the first tenant's cached validation result, and the same mutable result object. The separate live-validation cache is keyed only by XQL.

WHY IT MATTERS

Verification for one tenant can be presented to another. Changing policy or knowledge may leave old results active. This probe demonstrates cross-context reuse, not a confirmed disclosure of secrets or tenant data.

IMPROVEMENT PRINCIPLE

Separate reusable translation artifacts from tenant-specific execution evidence. Give each cache a complete identity, bounded lifetime and immutable values.

▶ **Source evidence**

Skip to content

HIGH REPRODUCED F03

Compiler review requirements disappear at the service boundary

A controlled engine result with requires_review=true and confidence=0.1 emerged from the service with confidence=0.75 and needs_review=false. Confidence is recalculated by strategy, while needs_review does not consult compiler diagnostics. Separately, the real compiler accepted a made-up field heuristically with confidence=0.9 and parity_rate=1.0.

WHY IT MATTERS

A translation can look safe even when fields are guessed or conditions require review. The compiler's parity_rate is a local heuristic, not measured agreement between Prisma and Cortex.

IMPROVEMENT PRINCIPLE

Use one typed result contract that retains provenance, unsupported constructs, dropped predicates and review requirements. Confidence should summarize evidence, not override it.

▶ **Source evidence**

Skip to content

HIGH REPRODUCED F04

Bulk translation drops encrypted credentials

The bulk handler decrypts the submitted credential payload but does not use the decrypted values. A request using the frontend’s credential contract reached the translation service with all three credential arguments null. The same route accepted and forwarded plaintext credential fields.

WHY IT MATTERS

Single, stream, v2 and bulk paths have different credential and validation behavior. Bulk users can believe live checks are enabled while requests run without their credentials.

IMPROVEMENT PRINCIPLE

Treat endpoint variants as transports around one request normalization and execution policy, including credential handling and validation status.

▶ **Source evidence**

Skip to content

HIGH SOURCE + INSTALL FAILURE F05

The shipped build and deployment contracts do not line up

Installing the backend’s pinned requirements failed on macOS at the unconditional pywin32==312 dependency; the backend Dockerfile installs that same file on Linux. Terraform requires backend authentication, but starts the frontend against localhost with no machine token. The client auto-mints identity tokens only for run.app URLs. CI also invokes docker compose build from the root although the compose file is under deploy/.

WHY IT MATTERS

The supplied image build has a platform dependency blocker, and the GCE frontend-to-backend path has an authentication mismatch as configured. These are repository findings; no live deployment was inspected or changed.

IMPROVEMENT PRINCIPLE

Keep one executable deployment contract per supported target, with platform-aware dependencies and an explicitly authenticated service-to-service connection.

▶ **Source evidence**

Skip to content

HIGH REPRODUCED F06

Automatic fine-tuning fails before submitting a job

load_gold_standard returns (examples, seen_rqls), but EphemeralSFTRunner treats the tuple as a list and adds hindsight examples. A dry run raises TypeError before any cloud operation. All 673 gold examples lack timestamp/added fields, so all are counted as new even after a future last-run marker.

WHY IT MATTERS

The advertised automatic training cycle cannot currently complete through this entry point. Its readiness accounting would also repeatedly count old examples after the loader bug is repaired.

IMPROVEMENT PRINCIPLE

Model training as a durable, idempotent job over a versioned dataset, with a committed watermark and an explicit job state.

▶ **Source evidence**

Skip to content

HIGH SOURCE CONFIRMED F07

The batch learning DAG lacks the required code and data handoffs

KFP components use python:3.11-slim and install third-party packages, but do not install this application. Several components import rql_xql or training and catch failures. The event input is a local JSONL path, and gold updates are local files rather than a dataset artifact passed between containers. Terraform creates a pull subscription but does not deploy the documented trigger consumer.

WHY IT MATTERS

The diagram describes an intended workflow, but the repository does not establish an end-to-end executable learning deployment. Ordering tasks does not share their files or application code.

IMPROVEMENT PRINCIPLE

Give batch workers versioned application images and explicit input/output artifacts. Make orchestration ownership, event delivery and failed-job visibility part of the architecture.

▶ **Source evidence**

Skip to content

HIGH REPRODUCED F08

“Blind” evaluation can still retrieve its reference answer

The integration fixture clears gold_standard but leaves _gold_index intact. A test translation still returned the gold-standard answer from an index of 652 normalized keys. EvalRunner accepts blind=true but does not pass a lookup/cache-disabling option to the service.

WHY IT MATTERS

Scores from these paths cannot be assumed to measure generalization. Gold examples are also used for in-context prompting and training, so benchmark lineage matters beyond disabling one cache.

IMPROVEMENT PRINCIPLE

Separate reference, training and held-out datasets by query family and provenance. Make evaluation isolation enforceable and report exactly which retrieval and model context were enabled.

▶ **Source evidence**

Skip to content

HIGH SOURCE CONFIRMED F09

Evaluation and model promotion are not a completed release gate

finetune.run_evaluation_and_alias constructs evaluation configuration, then explicitly simulates success and logs alias assignment. The preparation upload helper uploads train and validation files, not the test set referenced by that configuration. The standalone release gate uses lower thresholds than the documentation and can omit the live gate when its metric is absent.

WHY IT MATTERS

A completed tuning job is not evidence of a safe replacement model. This code does not actually promote a production alias; the risk is relying on a release guarantee that has not been implemented.

IMPROVEMENT PRINCIPLE

Keep training completion, independent evaluation and production selection as separate auditable decisions. A missing measurement should not satisfy a required gate.

▶ **Source evidence**

Skip to content

MEDIUM SOURCE CONFIRMED F10

Feedback and gold updates are local mutable state

Learning events append to local JSONL; gold examples are rewritten directly to YAML. Writes are not a transaction across events, gold, indexes and caches. save_to_knowledge_base catches write errors, while a caller can interpret its return as success. GCE uses a read-only container with temporary feedback storage; compose persists feedback but not the gold file.

WHY IT MATTERS

Multiple instances can disagree about knowledge, lose learning on restart or keep stale translations after a correction. Concurrent writers risk overwriting one another.

IMPROVEMENT PRINCIPLE

Use an authoritative, versioned knowledge store with durable events and explicit write outcomes. Separate proposed corrections from trusted reference material.

▶ **Source evidence**

Skip to content

MEDIUM REPRODUCED + SOURCE F11

Parity is a useful diagnostic, but not proof of equivalence

ResultMatcher returns 1.0 when both result sets are empty. It also uses normalized IDs and heuristic partial matching. A parity-loop gold-cache shortcut can return match_rate=1.0 without a fresh live comparison. Different learning paths accept different parity thresholds.

WHY IT MATTERS

No matching data, truncated result sets and heuristic identity matches can overstate semantic certainty. Tenant, time window, sample size and query limits affect the meaning of a match.

IMPROVEMENT PRINCIPLE

Attach the scope and strength of evidence to parity results. Distinguish no data, sample agreement and independently verified semantic coverage.

▶ **Source evidence**

Skip to content

MEDIUM REPRODUCED + SOURCE F12

Literal normalization and a prompt rule can change meaning

Gold lookup normalizes the entire RQL string to lowercase, including quoted values: case-distinct string literals collide. The canonical LLM system contract also states “contains → not contains”, reversing a predicate. The prompt defect is static; no model response was generated to measure its effect.

WHY IT MATTERS

These are semantic correctness risks even when the resulting query parses successfully.

IMPROVEMENT PRINCIPLE

Normalize syntax while preserving literal values, and use one tested semantic specification for compiler mappings, examples and prompts.

▶ **Source evidence**

Skip to content

MEDIUM REPRODUCED + SOURCE F13

Shadow and repair paths have interface mismatches

The shadow submission passes cc_url to a method expecting cc_api_url, so the background cognitive translation never runs in the focused probe. A separate repair call passes cc_api_key as cc_api_url. These branches are easy to miss in happy-path compiler tests.

WHY IT MATTERS

Shadow comparisons may silently disappear, while repair can target the wrong endpoint. The API-key-as-URL assignment also creates a potential logging exposure; no real key was used or leaked in this review.

IMPROVEMENT PRINCIPLE

Prefer typed request objects and narrow interfaces across optional branches, and make background failures observable.

▶ **Source evidence**

Skip to content

MEDIUM CONDITIONAL SOURCE FINDING F14

Authentication policy can be too permissive when incomplete

ID-token verification accepts an optional audience, and an empty principal allowlist permits any identity accepted by that verifier. Terraform leaves the audience empty when api_domain is unset. This is a configuration-dependent finding, not evidence of public access in a deployed environment.

WHY IT MATTERS

Requiring a signed identity is weaker than requiring the intended caller for this API. The frontend/backend authentication mismatch in finding 05 is a separate availability problem.

IMPROVEMENT PRINCIPLE

Make audience and authorized-caller policy explicit deployment invariants, with secure configuration validation.

▶ Source evidence

Skip to content

MEDIUM DESIGN ASSESSMENT F15

Operational ownership is blurred by large orchestration modules

The frontend is 2,467 lines, API server 1,213, pipeline translator 1,976, auto_fix 3,540 and XQL emitter 2,879. Validation, result mutation and credential logic repeat across endpoints. In-memory pending jobs and caches are per process; bulk timeouts stop waiting but do not cancel a running worker thread.

WHY IT MATTERS

Behavior depends on route, process and execution order. Scaling replicas does not make state durable or bound global work. The failing suite also contains shared-cache contamination and stale expectations.

IMPROVEMENT PRINCIPLE

Keep the modular monolith for synchronous translation, but sharpen module contracts and state ownership. Isolate batch learning by lifecycle; more microservices alone would add coordination costs.

▶ Source evidence

VALIDATION

The compiler suite passed; the broader service suite did not.

805 passed, 21 failed, 4 skipped across local unit and compiler tests. Compiler-specific: 283 passed, 4 skipped. Eleven focused probes reproduced the behaviors described in the guide. The full failure count includes stale tests and shared-cache contamination, so it is not a count of distinct defects.

No live tenant, cloud tuning, deployment or Docker-build validation was performed. The original lockfile install failed on pywin32; tests used a separate Python 3.13 dependency set. This review combined repository-wide indexing, detailed tracing of critical flows, configuration analysis and controlled local execution.

Skip to content

oes and limitations →

Download findings JSON ↓

Prepared from the checked-in project, not a live production environment. Source links open the private GitHub repository at the reviewed commit. Download inventory ·